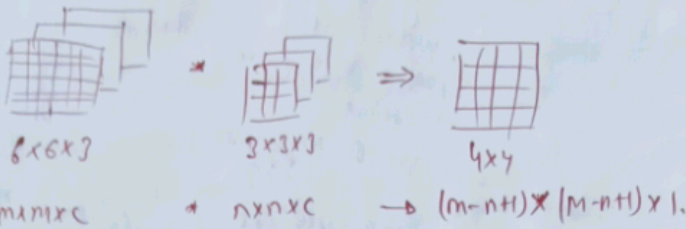


1. image shape  $n \times n$   $\otimes$  filter  $m \times m$   $\rightarrow$  feature map  $(n-m+1) \times (n-m+1)$

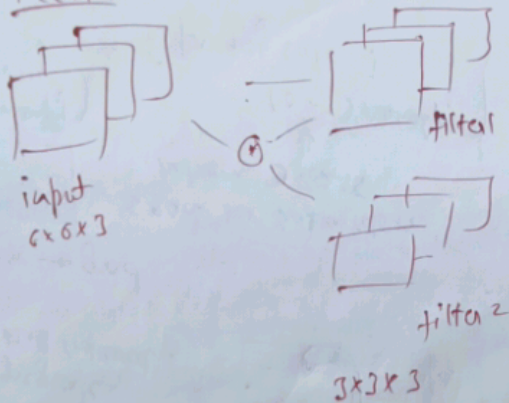
do  $n \geq m$ . # padding

padding  $\rightarrow$  image size reduce after convol  
 $\rightarrow$  center cell has more influence

for rgb image



no  $\downarrow$  in  $c \times m \times n$



5x5 image  $\otimes$   $3 \times 3 \Rightarrow$  5x5 image  
7x7 after padding

new formula

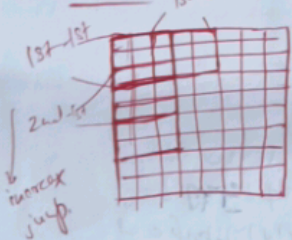
$$(n+2p-f+1)$$

$$= 5+2(1)-3+1$$

$$\Rightarrow 5$$

Two types of padding :-  
 $\rightarrow$  valid  
 $\rightarrow$  same

# Strides



stride  $\uparrow$  feature map size  $\downarrow$

$$\text{new formula } \left( \frac{n-f+2p}{s} + 1 \right)$$

increase stride

we? 1) just want high level feature  
2) computing fast

# Problem with Convolution:-

memory issue  
translation variance

Solution:-

1) increase stride  
2) pooling  $\rightarrow$  solve both problem

# translation variance



features shouldn't be location dependent  
so the model should be translation invariant

# pooling  $\rightarrow$  the image get translation invariance (location independent)

# Pooling

- max pooling
- min pooling
- avg pooling
- 2x pooling
- Global pooling

|   |   |   |   |
|---|---|---|---|
| 3 | 1 | 1 | 3 |
| 2 | 5 | 0 | 2 |
| 1 | 4 | 2 | 1 |
| 4 | 7 | 2 | 9 |

size  $\rightarrow$  (2,2)  
stride  $\rightarrow$  2  
type  $\rightarrow$  max

|   |   |
|---|---|
| 5 | 3 |
| 7 | 9 |

2x2 feature map

advantage

- 1) reduced size
- 2) translational invariance
- 3) Enhanced feature
- 4) No need of training

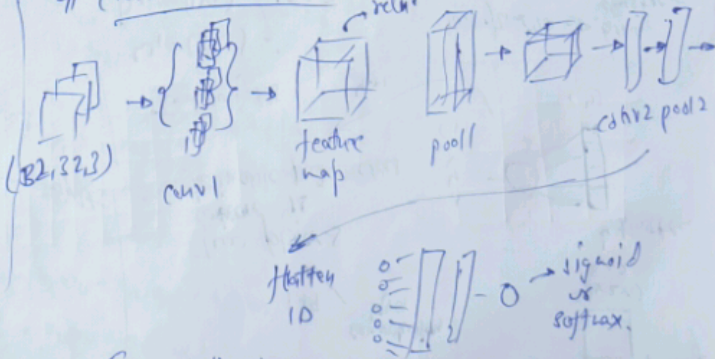
disadvantage

- 1) Image segmentation
- 2) lose a lot of information (75%)

(ImageNET)

- 1) LeNET
- 2) AlexNET
- 3) GoogleNET
- 4) Vgg NET
- 5) ResNET
- 6) Inception

# CNN architecture



# no. of convolution layers

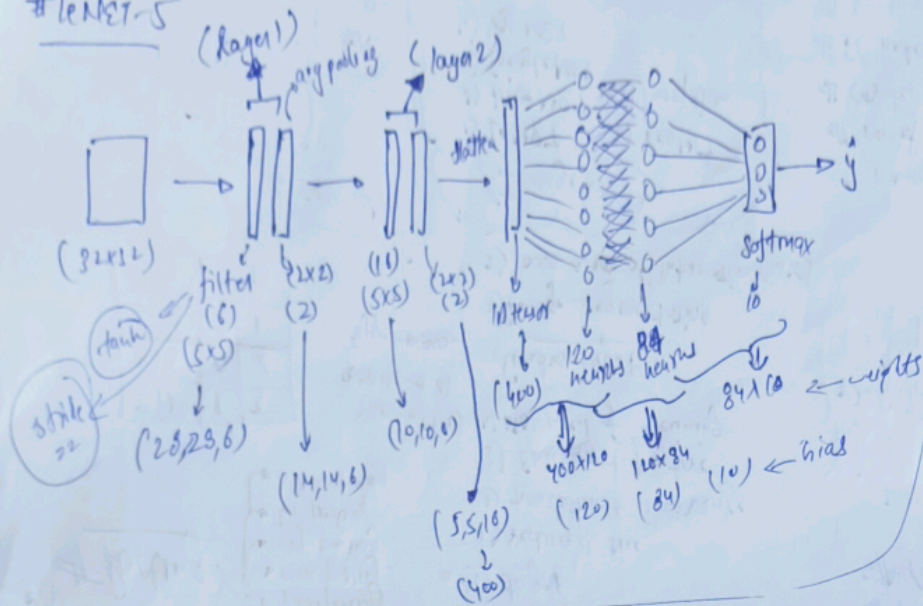
- # no. of filters
- # stride/padding
- # FC nodes
- # FC layers

- # activation
- # dropout
- # Batch norm

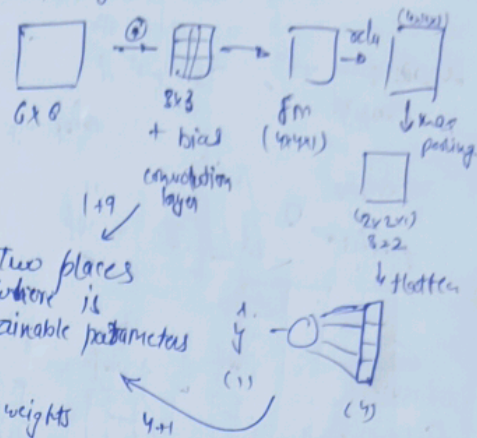
different architecture



# # LeNet-5



# # Backpropagation in CNNs:-



Two places where is trainable parameters

Total weights

Loss function - binary classification

$L = -y_i \log(\hat{y}_i) - (1 - y_i) \log(1 - \hat{y}_i)$

Formula  $z_1 = \text{Conv}(x, w_1) + b_1$

$A_1 = \text{relu}(z_1)$

$P_1 = \text{max pool}(A_1)$

$F = \text{flatten}(P_1)$

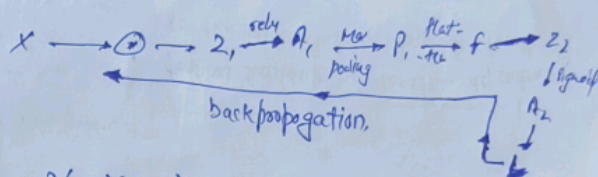
$z_2 = \text{Conv}(F, w_2) + b_2$

$A_2 = \text{softmax}(z_2)$

this is for forward propagation

Gradient descent (backpropagation):

$$\begin{aligned} w_1 &\leftarrow w_1 - \eta \frac{\partial L}{\partial w_1} \\ b_1 &\leftarrow b_1 - \eta \frac{\partial L}{\partial b_1} \\ w_2 &\leftarrow w_2 - \eta \frac{\partial L}{\partial w_2} \\ b_2 &\leftarrow b_2 - \eta \frac{\partial L}{\partial b_2} \end{aligned}$$



$$\begin{aligned} \frac{\partial L}{\partial w_1} &= \frac{\partial L}{\partial A_2} \times \frac{\partial A_2}{\partial z_2} \times \frac{\partial z_2}{\partial w_1} \\ \frac{\partial L}{\partial b_2} &= \frac{\partial L}{\partial A_2} \times \frac{\partial A_2}{\partial z_2} \times \frac{\partial z_2}{\partial b_2} \end{aligned}$$

# # Backpropagation in flattening

here  $\frac{\partial f}{\partial P_1} \rightarrow \text{reshape}(P_1)$  for discussing

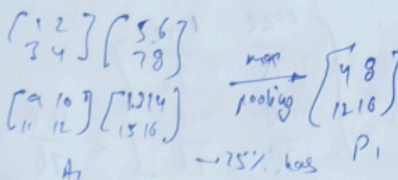
this doesn't have a quantity as it is non-measurable and is equally applied on everyone

$$\begin{aligned} \frac{\partial L}{\partial w_1} &= \frac{\partial L}{\partial A_2} \times \frac{\partial A_2}{\partial z_2} \times \frac{\partial z_2}{\partial F} \times \frac{\partial F}{\partial P_1} \times \frac{\partial P_1}{\partial A_1} \times \frac{\partial A_1}{\partial z_1} \times \frac{\partial z_1}{\partial w_1} \\ &= (A_2 - y) w_2 \cdot \text{reshape}(P_1, \text{shape}) \cdot \frac{\partial P_1}{\partial A_1} \times \frac{\partial A_1}{\partial z_1} \times \frac{\partial z_1}{\partial w_1} \end{aligned}$$

# # Backpropagation for convolution

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial A_2} \times \frac{\partial A_2}{\partial z_2} \times \frac{\partial z_2}{\partial F} \times \frac{\partial F}{\partial P_1} \times \frac{\partial P_1}{\partial A_1} \times \frac{\partial A_1}{\partial z_1} \times \frac{\partial z_1}{\partial w_1}$$

# # Backpropagation in Maxpooling



$$\frac{\partial L}{\partial P_1} = \begin{bmatrix} \frac{\partial L}{\partial A_2} & \frac{\partial L}{\partial z_2} \end{bmatrix}$$

convolution from 2x2 to 4x4 is back derivative

why zero?

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial A_2} \times \frac{\partial A_2}{\partial z_2} \times \frac{\partial z_2}{\partial F} \times \frac{\partial F}{\partial P_1} \times \frac{\partial P_1}{\partial A_1} \times \frac{\partial A_1}{\partial z_1} \times \frac{\partial z_1}{\partial w_1}$$

$$\begin{aligned} z_{11} &= w_{11}w_{11} + w_{12}w_{12} + w_{21}w_{21} + w_{22}w_{22} + b_1 \\ z_{12} &= w_{11}w_{11} + w_{12}w_{12} + w_{21}w_{21} + w_{22}w_{22} + b_1 \\ z_{21} &= w_{11}w_{11} + w_{12}w_{12} + w_{21}w_{21} + w_{22}w_{22} + b_1 \\ z_{22} &= w_{11}w_{11} + w_{12}w_{12} + w_{21}w_{21} + w_{22}w_{22} + b_1 \end{aligned}$$



$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z_1} \frac{\partial z_1}{\partial w_1}$$

$$\frac{\partial L}{\partial w_1} = \left[ \frac{\partial L}{\partial w_{11}} \quad \frac{\partial L}{\partial w_{12}} \right]$$

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial z_{11}} \times \frac{\partial z_{11}}{\partial w_{11}} + \frac{\partial L}{\partial z_{12}} \times \frac{\partial z_{12}}{\partial w_{11}} + \frac{\partial L}{\partial z_{21}} \times \frac{\partial z_{21}}{\partial w_{11}} + \frac{\partial L}{\partial z_{22}} \times \frac{\partial z_{22}}{\partial w_{11}} \quad (\text{similarly})$$

$$\frac{\partial L}{\partial w_1} = \text{conv} \left( x, \frac{\partial L}{\partial z_1} \right)$$

$$\frac{\partial L}{\partial z_1} = \begin{bmatrix} \frac{\partial L}{\partial z_{11}} & \frac{\partial L}{\partial z_{12}} \\ \frac{\partial L}{\partial z_{21}} & \frac{\partial L}{\partial z_{22}} \end{bmatrix}$$

# Why use Pretrained model?

→ Data hungry → image data + labeled

Well labelled

# ImageNet

2001 → Fei Fei Li → 1.4 million images → 2000 categories

↓  
bounded label too

# Data Augmentation

Generate data by transforming the available data

Advantages

→ increase data  
→ reduce overfitting

Image Recognition Competition

→ after two year AlexNET made a breakthrough with his CNN model and won and revolution in 2012.

2012 AlexNET → 16.4%  
2013 ZFNET → 11.7%  
2014 VGG → 7.3%  
2015 GoogleNET → 6.7%  
2016 ResNET → 3.5%

# Transfer Learning :- Instead of training a model from scratch, use a pre-trained model and fine-tune it for your specific task.

eg:- let say you want to make a classifier which can classify between men and <sup>women</sup> ~~human monkey~~ and you already have a model that can differentiate between monkeys and humans.

the primitive layer that extract edges, face shape, ears, nose, hairs can be used in our model and no need to train again.

Using transfer learning will be beneficial here.

Advantages

- ① Reduced training time
- ② Require less data.
- ③ Improve performance (for small datasets)

④ Leverages learned features (eg, edges, texture, pattern, etc)

Working

→ the early layers are frozen which have primitive features

→ the final layers are trained on our own data sets.

→ Popular transfer Learning Models

- Computer Vision : ResNet, VGG, Inception
- NLP : BERT, GPT, ROBERTa, T5



## # Problem with Sequential Model

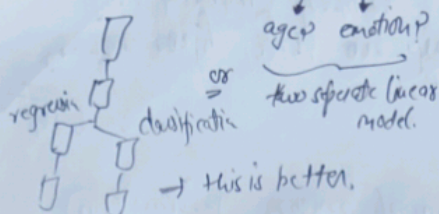
ANN  $\leftrightarrow$  CNN

↳ Sequential - keras

- ↳ 1 input
- ↳ 1 output
- ↳ linear

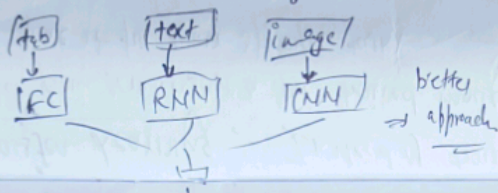
Eg where linear model fails

1) image dataset  $\rightarrow$  2800 image



2) product meta data, product description, product image  
(tabular) (textual) (image)

predict price?



## # Functional API $\hookrightarrow$ Can be used to define the flow of data. (God level knowledge) $\hookrightarrow$ more flexible as you are not restricted for sequential model.

### Common use cases

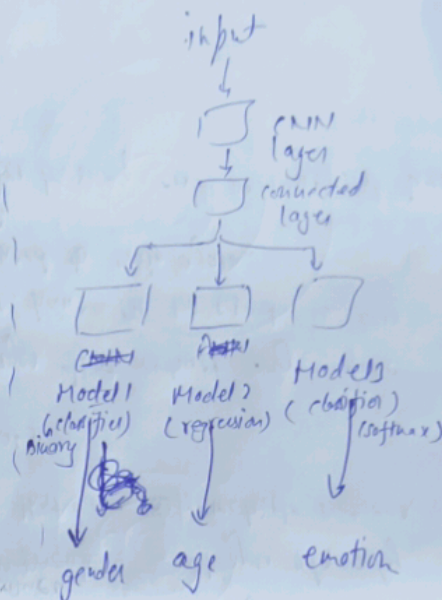
- ↳ Multi task models
- ↳ Residual networks (ResNet)
- ↳ Attention mechanisms
- ↳ Autoencoders
- ↳ Custom Research models

eg: lets say you want to develop a model that classify, ~~ages~~ age/emotion/grades of a photo.

one way to do this is use three different models

disadvantage, primitive features are training again and again

$\hookrightarrow$  Can go for a model like



## # RNN Networks Recurrent Neural

is a type of sequential model to work on sequential data

- $\hookrightarrow$  they have memory feature (means can remember previous input)
- $\hookrightarrow$  which is why it works good on sequential data.

Data for RNN (timestamp, input-features)

reviews

$[[10000], [01000], [00100]]$

(3,5)  $\leftarrow$  # of features  
no. of timestamps

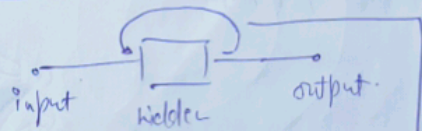
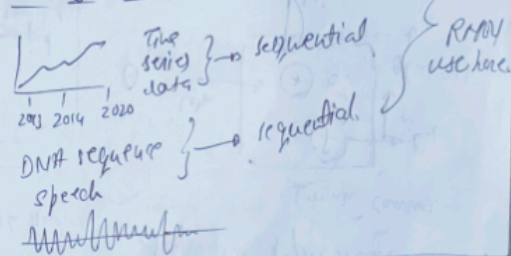
| Review               | Sentiment |
|----------------------|-----------|
| $x_1$ movie was good | 1         |
| $x_2$ movie was bad  | 0         |
| $x_3$ movie was good | 0         |

$[1,0000]$  movie  $[01000]$  was  $[00100]$  good  
 $[00010]$  bad  $[10001]$  not

for eg: iq / mark / gender / Place  $\rightarrow$  non sequential  
as if it is mark / iq / gender / places its the same thing

(MLP)

eg: Hi My Name is Ad  $\rightarrow$  sequential



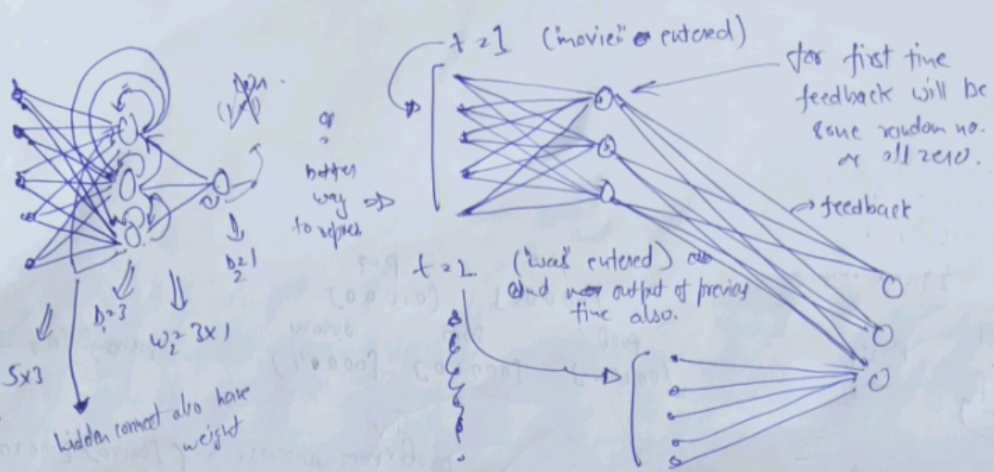
{ time basis }

$t=1 \rightarrow h_1$   
 $t=2 \rightarrow h_2$   
 $t=3 \rightarrow h_3$   $\rightarrow$  feed on basis of time

(feedback)

$\hookrightarrow$  this is what makes RNN unique



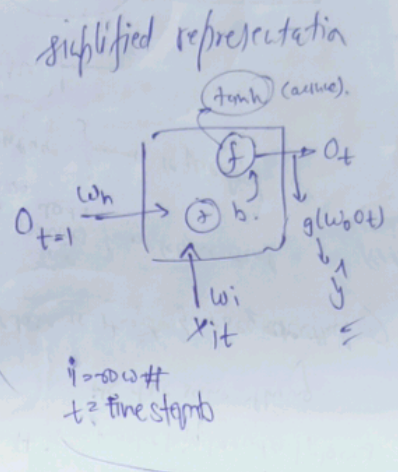
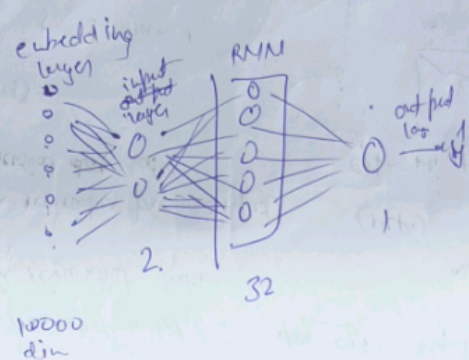


| review                           | sentiment |
|----------------------------------|-----------|
| $x_{11}, x_{12}, x_{13}$         | 1         |
| $x_{21}, x_{22}, x_{23}$         | 0         |
| $x_{31}, x_{32}, x_{33}, x_{34}$ | 0         |

RNN layers  $27 \times 3 + 9$   
 dense layers  $4 \times 27$   
 total = 81

forward propagation  $\rightarrow$  unfolding through time

Integer encoding  
 zero for padding  
 "How are you?"  $\rightarrow [1, 2, 0]$   
 "I am fine"  $\rightarrow [3, 4, 5]$

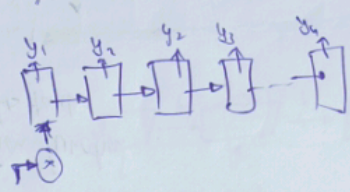


# # Types of RNN:-

- Many to one
- One to many
- Many to many  $\rightarrow$  same / Variable
- One to one

## (2) One to Many

output  $\rightarrow$  non sequential: image, video  
 input  $\rightarrow$  non sequential: image, video  
 output  $\rightarrow$  sequential: time series, words



eg:- image captioning.  $\rightarrow$  caption output

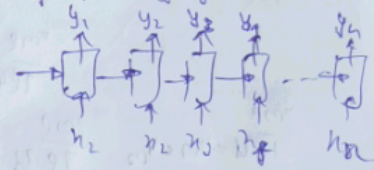
## (3) Many to Many

input  $\rightarrow$  sequential  $\rightarrow$  seq 2 seq  
 output  $\rightarrow$  sequential

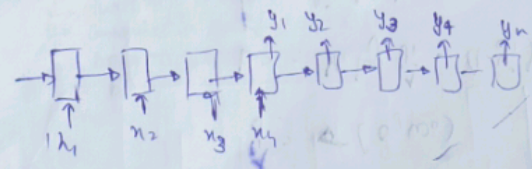
Two types  $\rightarrow$  same length / variable length

(input seq  $\rightarrow$  output seq)  $\rightarrow$  length

eg: pos tagging  $\rightarrow$  NLP

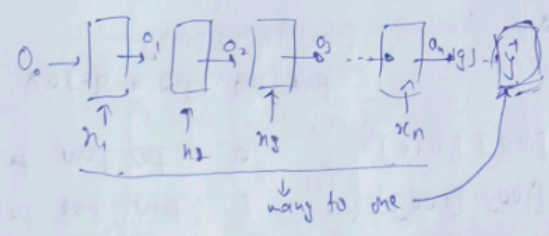


variable length egr  
 machine translation (google translate)



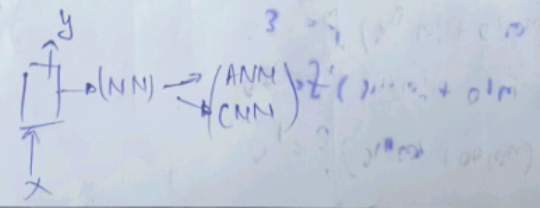
## (1) Many to one

input  $\rightarrow$  sequence: sent, charactrs, timeseries  
 output  $\rightarrow$  non sequential: int, colors, classes  
 eg: sentiment analysis, output  $\rightarrow$  {0, 1}  
 Rating prediction, output  $\rightarrow$  (1-5)



## (4) One to one

output  $\rightarrow$  non-sequential  
 input  $\rightarrow$  non-sequential  $\rightarrow$  image classifier (example)



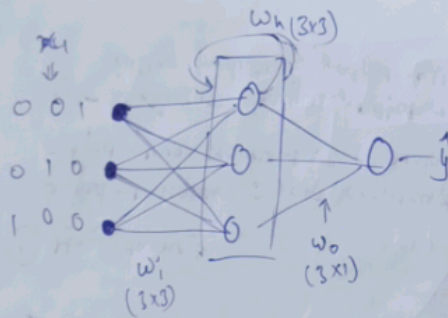


# # Back Propagation in RNN

↳ also known as BP in time.

| text        | y | X                       | y |
|-------------|---|-------------------------|---|
| cat mat rat | 1 | $x_1 [100] [010] [001]$ | 1 |
| rat rat mat | 1 | $x_2 [001] [001] [010]$ | 1 |
| mat mat cat | 0 | $x_3 [010] [010] [100]$ | 0 |

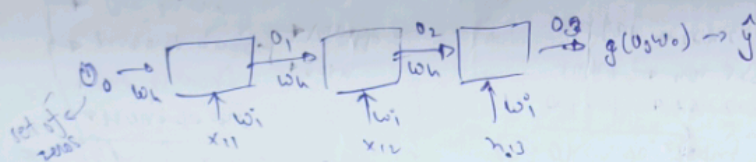
vocab  $\rightarrow$  cat, rat, mat  
 $[1,0,0], [0,1,0], [0,0,1]$



firstly input first sentence word by word.

and each word representation occur.

Activation function  
 $\hookrightarrow$  tanh



$$o_1 = f(n_1 w_1 + o_0 w_0)$$

$$o_2 = f(n_2 w_1 + o_1 w_0)$$

$$o_3 = f(n_3 w_1 + o_2 w_0)$$

$$\hat{y} = \sigma(o_3 w_0)$$

$$L = -y_i \log \hat{y}_i - (1-y_i) \log (1-\hat{y}_i)$$

gradient descent  $w_i, w_0, w_0$

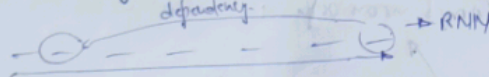
$$w_1 = w_1 - \eta \frac{\partial L}{\partial w_1} \quad w_0 = w_0 - \eta \frac{\partial L}{\partial w_0}$$

$$w_0 = w_0 - \eta \frac{\partial L}{\partial w_0}$$

can also add bias

## # Problem with RNN?

- $\rightarrow$  problem of long term dependency
- $\rightarrow$  problem of ~~unstable~~ gradient stagnated training.



# timestamp

if it is too long, it forget the ~~most~~ earlier one.

it arises from vanishing gradient.  
 + the problem of long term dependency

$$\frac{\partial L}{\partial w_{in}} = \left[ \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_{in}} + \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_2} \frac{\partial w_2}{\partial w_{in}} \right]$$

$$+ \left[ \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_3} \frac{\partial w_3}{\partial w_1} \frac{\partial w_1}{\partial w_{in}} \right] \rightarrow \text{long term dependency}$$

if timestamp of 100  $\rightarrow$  every term  $< 0.1$  due to tanh.

$$\left[ \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial w_0}{\partial w_{in}} \frac{\partial w_0}{\partial w_{in}} - \frac{\partial y}{\partial w_{in}} \right] \approx 0$$

also in case of relu it will face exploding gradient.

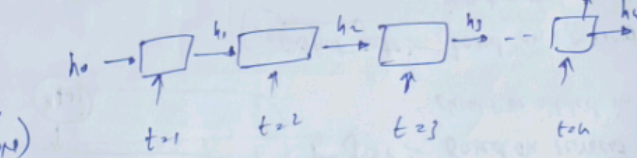
## # LSTM Core Idea

works on how human brain works using long term and short term & context.

save important thing and remove non-important things.

LSTM

STM (simple RNN)



LSTM

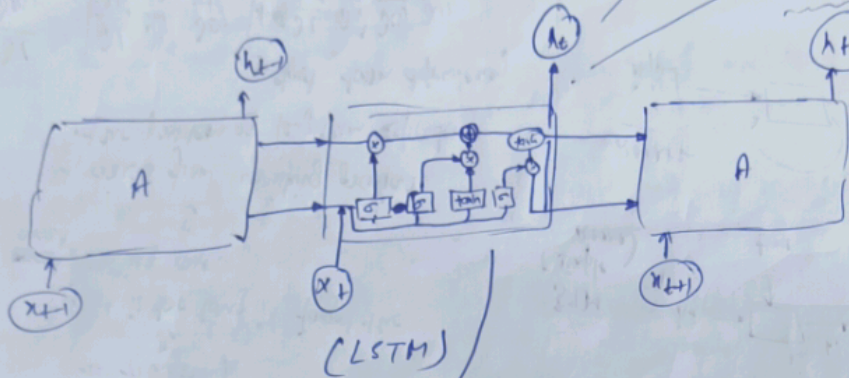
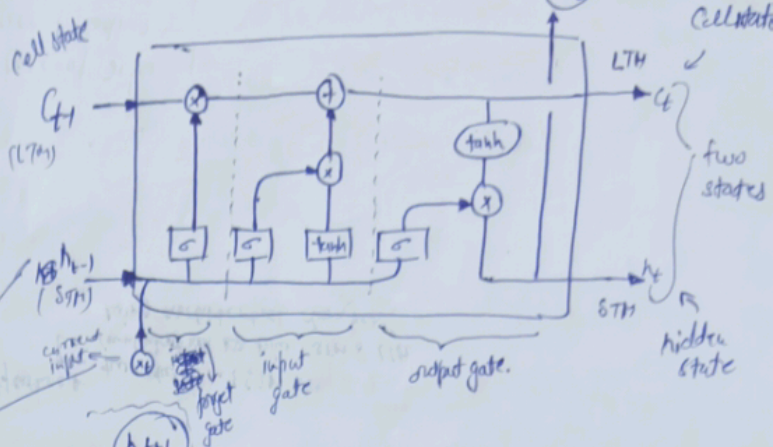
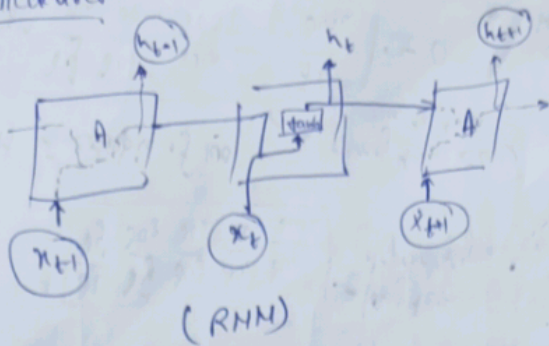


Two difference  $\rightarrow$  two states in LSTM  
 $\hookrightarrow$  communication b/w STM & LSTM  
 bring architectural changes





# # architecture



forget gate:- decide what to remove from long term memory.

input gate:- Based on current input, what should be added in long term memory.

output gate:- based on current input from input gate, output gate decide the output that passes in next state. Short term memory.

two main thing happens!

- 1) update the LTM
- 2) create the STM.

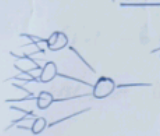
## # The gates

- forget gate
- input gate
- output gate

Neural Network layer  
 $\sigma$  -> same in all.  
 $\tanh$  -> same in all.

all have same dimensions as  $C_t$  and  $h_t$

$f_t$  forget gate  
 $i_t$  input gate  
 $o_t$  output gate  
 candidate cell state



all have activation function sigmoid.

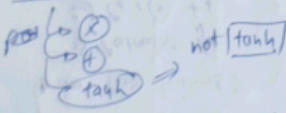
this now itself is a hyperparameter.

$f_t, i_t$  -> work as filters

$\bar{C}_t$  -> new potential information that can be stored

same dimensions ->  $h_t, C_t$   
 vectors  
 $[0.1, 0.3, 0.9]$   
 (3d vector)

point wise operations:-



$$C_{t-1} = [4, 5, 6] \quad f_t = [1, 2, 3]$$

$C_{t-1} \otimes f_t$  -> element wise product  
 $C_t = [4, 10, 18]$

sentiments -> vectorization ONE  
 find no. of unique  
 assign values  
 write in format no.s/0,1.

similarly point wise  $+$  &  $\tanh$

same dimension of incoming and outgoing data

|     |     |     |   |
|-----|-----|-----|---|
| Cat | rat | mat | 0 |
| at  | rat | rat | 0 |
| mat | mat | cat | 1 |

$$[100] \quad [010] \quad [001]$$

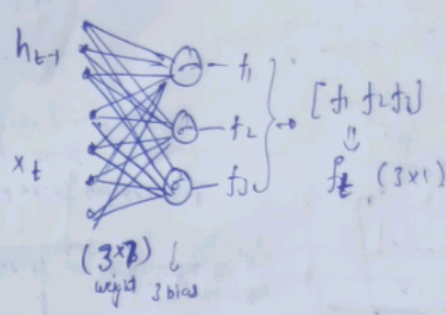
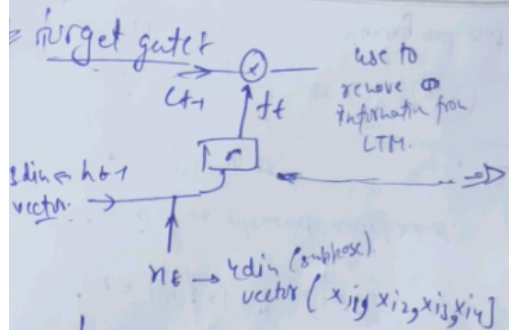
$$t=1 \quad t=2 \quad t=3$$

$$C_t \rightarrow A \rightarrow A \rightarrow A$$

$$t=1 \quad t=2 \quad t=3$$

these three are same with (recurrence)





$f_t \rightarrow [0, 0, 0] \rightarrow 100\%$  remove

$[1, 1, 1] \rightarrow 0\%$  remove

as  $C_{t-1} \otimes [0, 0, 0] = (0, 0, 0)$

$C_{t-1} \otimes [1, 1, 1] = C_{t-1}$

do two things:-

- 1) Calculate  $f_t$
- 2)  $C_{t-1} \otimes f_t$

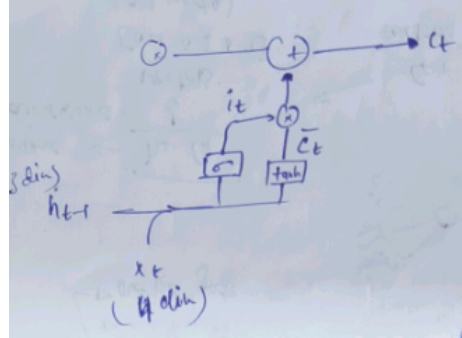
$$f_t = \sigma(w_f [h_{t-1}, x_t] + b_f)$$

$(3 \times 1)$   $(7 \times 1)$   $3 \times 1$   $3 \times 1$

that's why we call it gate

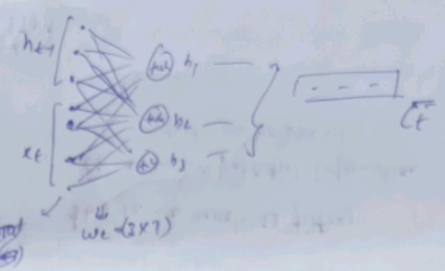
$f_t$  has power how much information of LTM goes forward.

### # Input gates



it works in 3 stages:-

- 1) Calculate  $i_t$  and  $C_t$
- 2) Calculate  $i_p$  to decide what values are important to add in LTM.
- 3)  $C_t = f_t \otimes C_{t-1} + i_p \otimes C_t$



it has two Neural networks

pointwise operations:-

$$\text{new LTM} \leftarrow C_t = i_p \otimes C_t + f_t \otimes C_{t-1}$$

filter candidate cell state

previous cell state after forgetting.

if  $f_t = [1, 1, 1]$

$i_t \otimes C_t = [0, 0, 0]$

$C_{t-1} \rightarrow C_t$

long term information can be retained.

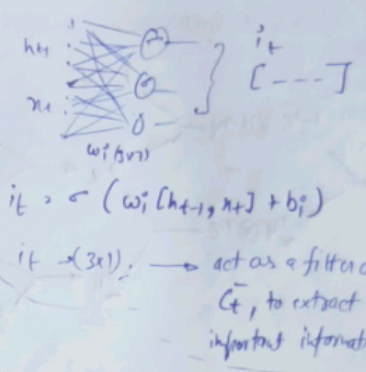
potential important information

$$C_t = \tanh(w_c [h_{t-1}, x_t] + b_c)$$

$(3 \times 1)$   $(7 \times 1)$   $3 \times 1$

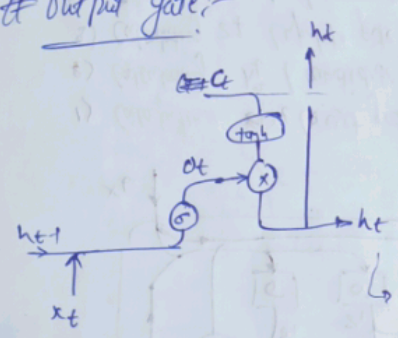
$C_t \rightarrow 3 \times 1$

similarly second n.n.



This is all working because of multiple neural networks used on different purpose to calculate the values of filters that do magic.

### # Output Gate:-

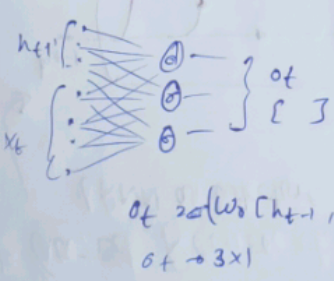


steps:-

- 1) element wise tanh on  $C_t \rightarrow [-1, 1]$  range
- 2) calculate  $o_t$  (filter) from  $h_{t-1}$  and  $x_t$
- 3) filter the tanh( $C_t$ ) with  $o_t$  and comes with a STM to pass  $h_t$ .

$$h_t = o_t \otimes \tanh(C_t) \rightarrow (3 \times 1)$$

Thus output gate filters the LTM to extract the recent data and form short term memory with it and the input that is given



### # Next word predictor / text generator

data:- Hi my name is Anand  
I live in delhi

So have to write it / supervised

input/output

| input               | output           |
|---------------------|------------------|
| Hi                  | my               |
| Hi my               | name             |
| Hi my name is       | live             |
| Hi my name is Anand | I live in delhi. |

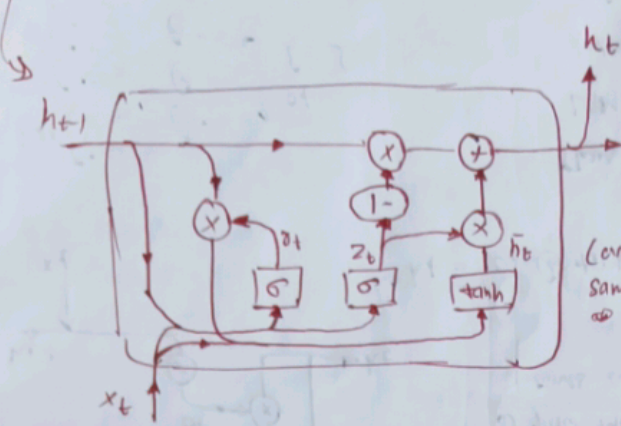


# # Gated Recurrent Unit:-

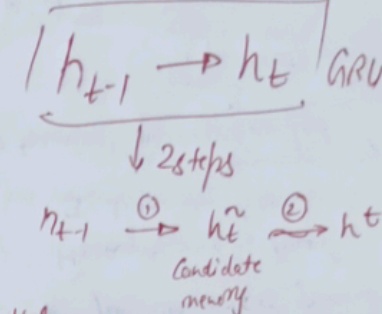
(RNN architecture)

came because it has simpler architecture than LSTM, with 2 gate and 1 state

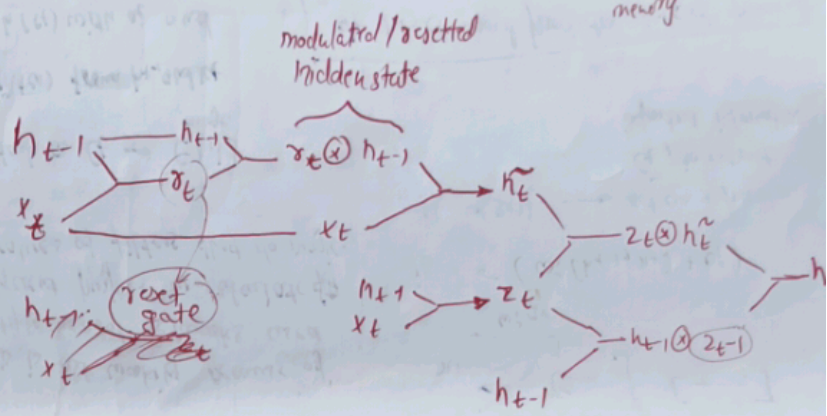
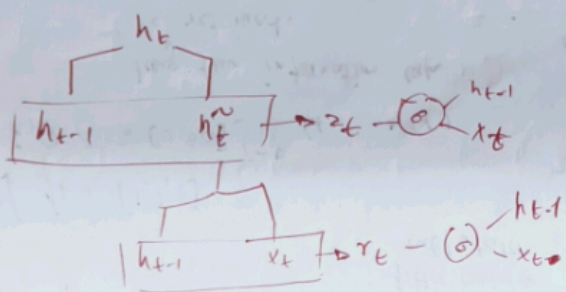
after this it has less parameters and training is fast, and it gives performance comparable to LSTM (sometimes someone wins 50-60)



same dimension  $h_{t-1}, h_t, r_t, z_t, \tilde{h}_t, x_t$   
 candidate hidden state  
 neural networks (fully connected)  
 every  $\square$  has same dimensions  
 $\circ$  point wise (element wise)



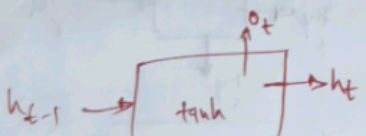
- 1) Calculating  $r_t$  (reset gate)
- 2) Calculating  $\tilde{h}_t$  (candidate hidden gate)
- 3) Calculating  $z_t$  (update gate)
- 4) Calculate  $h_t$  (current hidden state)



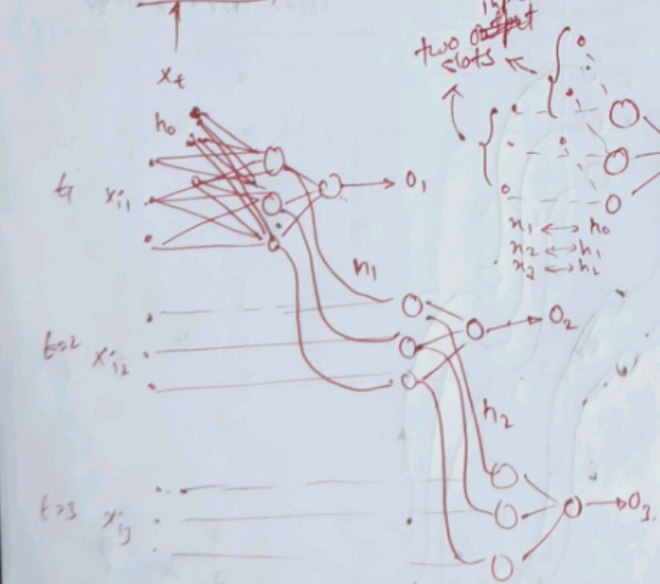
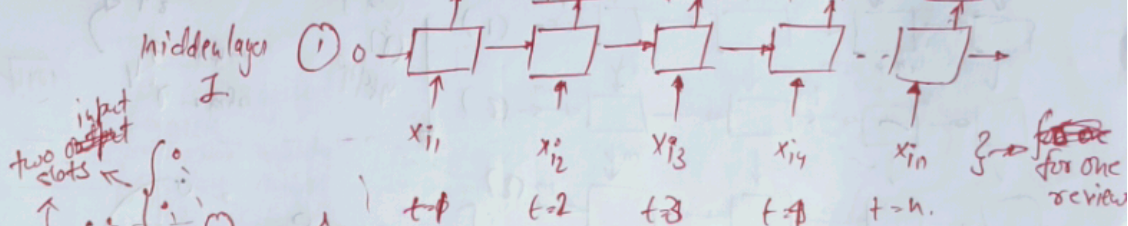
MATHEMATICA FLOWCHART OF GRU.

# # Deep RNN:-

we know Rnn look like:-

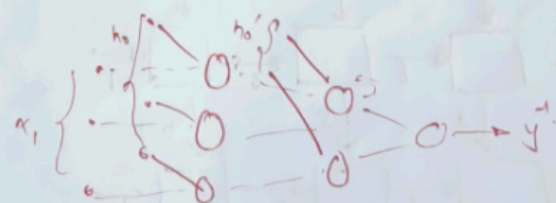
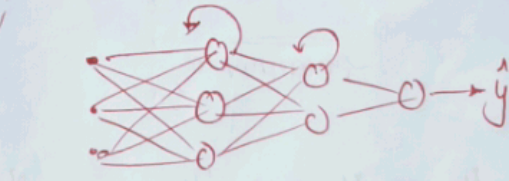


in deep RNN:-  
 hidden layer 2  
 hidden layer 1

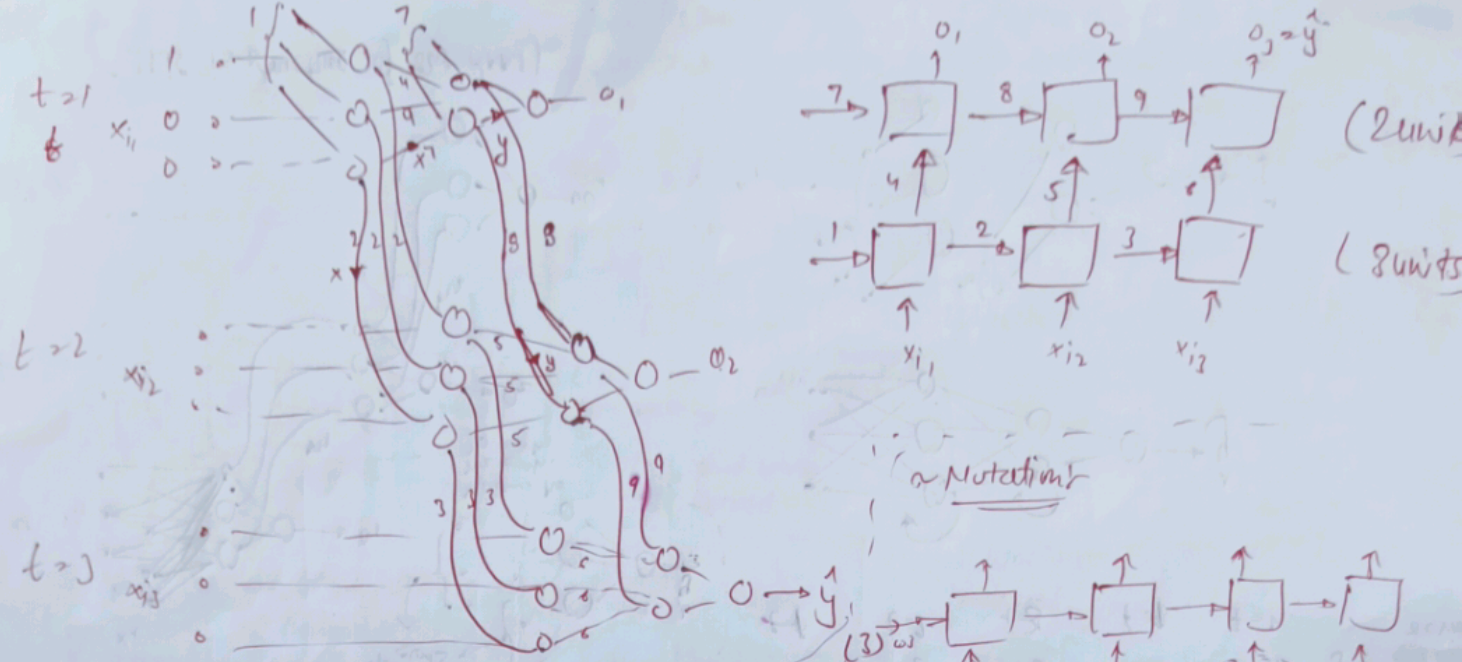


(this is structure of one Rnn)

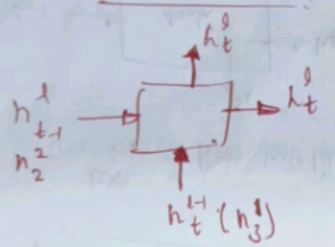
it just like we are stacking our RNN  
 helps to solve complex problem







circled RNN cell



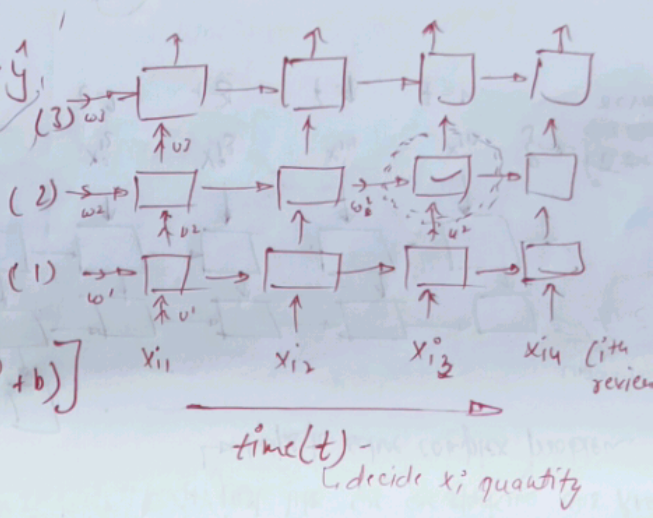
Definition

$h_t^l \rightarrow$  layer  
 $h_t \rightarrow$  time  
 $h_3^2$

depth  
 $(l)$   
 $(\text{layer})$

$$\therefore h_t^{(l)} = \tanh(w^l h_{t-1}^{(l)} + u^l h_t^{(l-1)} + b)$$

Notation



When to use Deep RNN

- Complex tasks of speech recognition, machine translation
- Large datasets of overfitting?
- Sufficient computational resources.

# Bidirectional RNN -  $y_1, y_2, y_3, y_4$

BiLSTM zeros

Application

- POS tagging
- NER
- Machine Translation
- Sentimental Analysis
- Time Series Analysis

Drawback

- Complexity
- Overfitting

$$\vec{h}_t = \tanh(\vec{w} \vec{h}_{t-1} + \vec{u} x_t + \vec{b})$$

$$\overleftarrow{h}_t = \tanh(\overleftarrow{w} \overleftarrow{h}_{t+1} + \overleftarrow{u} x_t + \overleftarrow{b})$$

$$y_t = \sigma(v[\vec{h}_t, \overleftarrow{h}_t] + b)$$



# # Seq 2 Seq Models → tasks and its type

↳ RNN → sequential

## Types of RNN

(1)



many to one.  
eg- sentiment analysis  
rating.

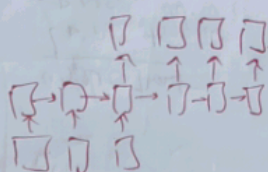
(3) Many to Many

(i) synchronous



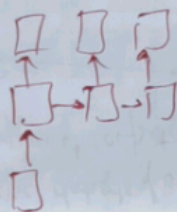
eg- speech tagging  
named Entity Recognition (NER)

(ii) asynchronous



eg- Machine translation

(2)



one to many (input is  
scalar, non sequential)  
eg- Image Captioning

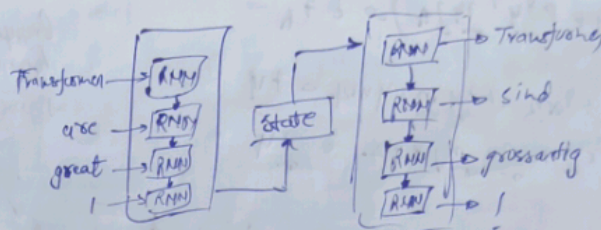
## Seq 2 Seq Tasks:-

- 1) text summarization
- 2) Question Answer
- 3) Chat bots
- 4) Speech to texts

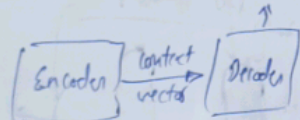
## History

Encoder → Attention → Transformer → Transfer learning  
Decoder

Encoder → process word by word, and compress  
Decoder → now bring not just word by word



## # Encoder / Decoder:-



eg- Eng

Think about it  
care in

Hindi

चिन्ता में  
सावधानी  
<start> <end>

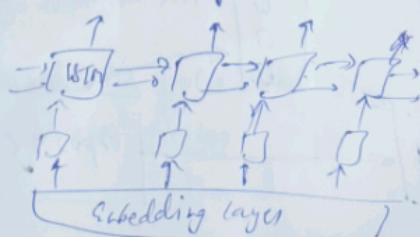
[1000000]  
[0, 1, 0, 1, 0, 1, 0, 1, 0, 1]

loss calculation

→ categorical cross entropy

$$L = - \sum_{i=1}^n y_i^{true} \log(y_i^{pred})$$

Improvement:- Embedding



→ one hot encoding → dimension increase  
(1 label)

Eng

[10000]  
[01000]  
[00100]  
[00010]  
[00001]

Hindi

[1000000]  
[0100000]  
[0010000]  
[0001000]  
[0000100]  
[0000010]  
[0000001]

improvement 2:- Recurrent RNN  
improvement 2:- Recurrent RNN

→ 333

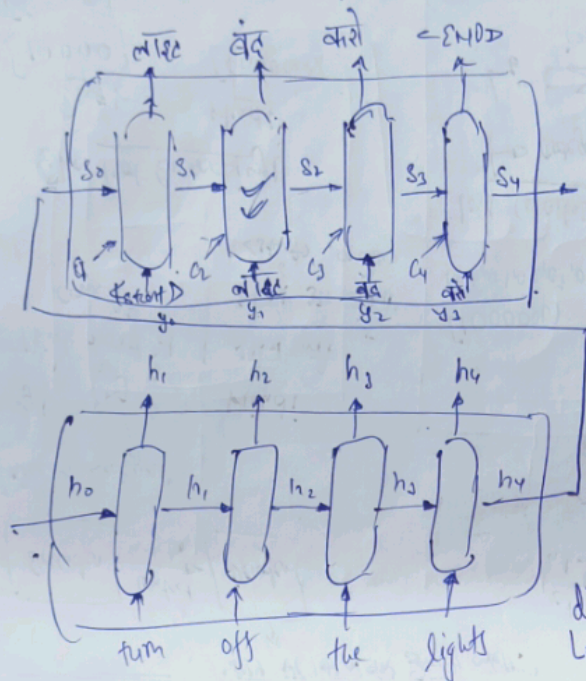
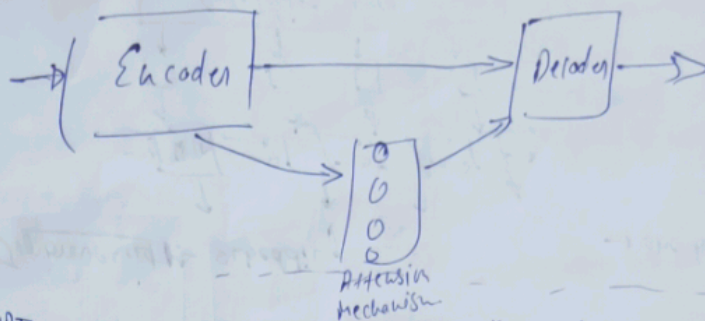


# # Attention Mechanism :-

Problem with En (de) :-

- Can't comprehend large paragraphs.
- inconsistent results

What is attention mechanism



Previously we were spreading

$[s_1, y_1]$   $i=2$

now we will send one more attention information from  $h_1$  to  $h_4$  which is useful one.

$c_i \rightarrow$  attention input.

$[s_1, s_2, c_2]$   $i=2$ .

$c_i \rightarrow$  scalar  $\times$

$\rightarrow$  vector

$\rightarrow$  matrix  $\times$

dimension  $\rightarrow$  (padding of  $h_i$ )

if  $h_1, h_2$  two vectors,  $\downarrow$  weighted sum.

# alignment score  $r$

$(\alpha_1, \alpha_2, \alpha_3, \alpha_4) \rightarrow c$

$$c = \alpha_1 h_1 + \alpha_2 h_2 + \alpha_3 h_3 + \alpha_4 h_4$$

correct notation

$$c_i = \sum_j \alpha_{ij} h_j$$

$j \rightarrow$  no. of hidden state  
 $i \rightarrow$  ith hidden state

total  $\alpha \rightarrow i \times j \rightarrow n^2$

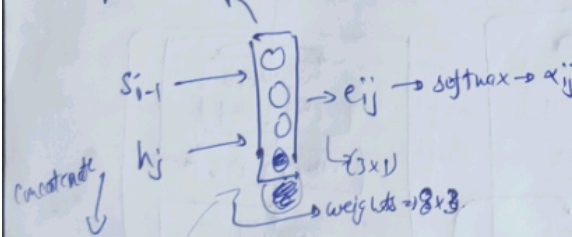
$\alpha_{21} \rightarrow$  alignment depends upon  $h_1, s_1$

## # Bahdanau Attention

$$\alpha_{ij} = f(h_j, s_{i-1})$$

approximate  $\leftarrow$  instead of func use a neural network

feed forward neural network.



|          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|
| $s_{01}$ | $s_{02}$ | $s_{03}$ | $s_{04}$ | $h_{11}$ | $h_{12}$ | $h_{13}$ | $h_{14}$ |
| $s_{01}$ | $s_{02}$ | $s_{03}$ | $s_{04}$ | $h_{21}$ | $h_{22}$ | $h_{23}$ | $h_{24}$ |
| $s_{01}$ | $s_{02}$ | $s_{03}$ | $s_{04}$ | $h_{31}$ | $h_{32}$ | $h_{33}$ | $h_{34}$ |
| $s_{01}$ | $s_{02}$ | $s_{03}$ | $s_{04}$ | $h_{41}$ | $h_{42}$ | $h_{43}$ | $h_{44}$ |

it changes for  $s_1, s_2, s_3$

this is not changing

different  $\alpha$ 's will be there

## # Luong Attention

$\alpha_{ij} \propto s_i^T h_j \rightarrow$  dot product.

$s_i = [a, b, c, d]$

$h_j = [e, f, g, h]$

$$s_i^T h_j = a \cdot e + b \cdot f + c \cdot g + d \cdot h$$

scalar  $e_{ij}$

$\downarrow$  softmax.

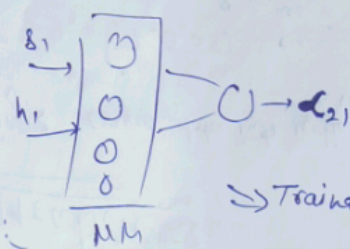
$\downarrow$   $\alpha_{ij}$

$\rightarrow$  doesn't have extra weight and training.  
 $\rightarrow$  performed better.

Luong attention is also called multiplication attention.

$(\alpha_{ij} \rightarrow h_j, s_{i-1})$

to calculate the importance of any  $h$  we also need to consider the completed translation  $s_{i-1}$ .



$\Rightarrow$  Trainable

$$\begin{cases} s_1 h_1 \rightarrow e_{11} & s_1 h_2 \rightarrow e_{12} \\ s_1 h_3 \rightarrow e_{13} & s_1 h_4 \rightarrow e_{14} \end{cases}$$

$$\alpha_{11} = \frac{e_{11}}{e_{11} + e_{12} + e_{13} + e_{14}}$$

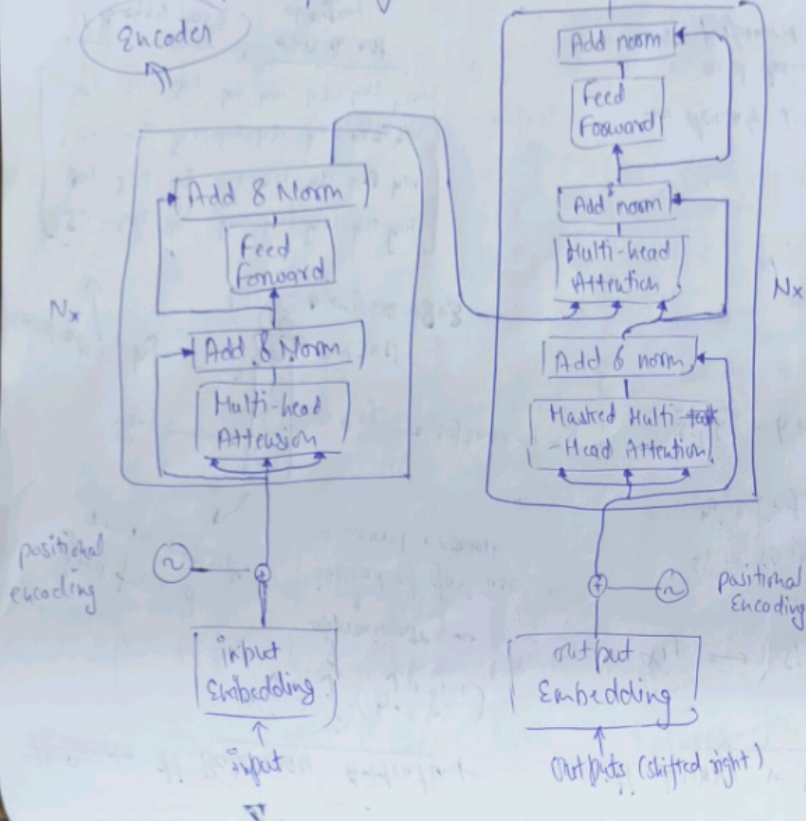
$$c_i = \alpha_{11} h_1 + \alpha_{12} h_2 + \alpha_{13} h_3 + \alpha_{14} h_4$$

$s_i, c_i$  important as it concatenate with output.



# # Transformer:-

- a kind of ~~art~~ neural network architecture
- used in seq 2 seq task
- support parallel processing



1) It don't have LSTM/RNN (as they can't help in parallel processing and fine tuning).

↳ based on self attention.

## Advantages

- Scalability
- Transfer Learning
- Multiple ip/op
- flexible architecture
- ecosystem
- Logging base
- Integrated easily.

## Disadvantages

- High computation
- Data (supervised)
- Overfitting
- Energy consumption
- Interpretability
- Bias
- Ethical concern

## # Self Attention

→ word Embedding: convert data into uniform vector

king: [0.9, 0.1, 1, 0, 0.9] every dim has a meaning that we don't know.  
queen: [0.9, 0.2, 0.4, 1, 0]

problem "Average meaning"

for e.g. Apple makes good phone  
give weights to both quality ← Apple is good for health

also these values are static thus can't be modify now after it was assigned.

got [Apple launched a new phone while I was eating an orange]

don't consider it as fact also because there is orange and word eating.

has to be dynamic based on contextual understanding

To solve it there comes concept of Self attention

## Example:-

money bank, grows / river bank, flows

this is different

this is different.

bank<sub>1</sub> → bank<sub>1</sub>x or bank<sub>2</sub>y or  $\frac{bank_1 + bank_2}{2}x$   
bank<sub>1</sub> → 0.2 money + 0.7 bank + 0.1 grows  
bank<sub>2</sub> → 0.2 river + 0.7 bank + 0.1 flows

now bank<sub>1</sub> ≠ bank<sub>2</sub> it depends upon context?

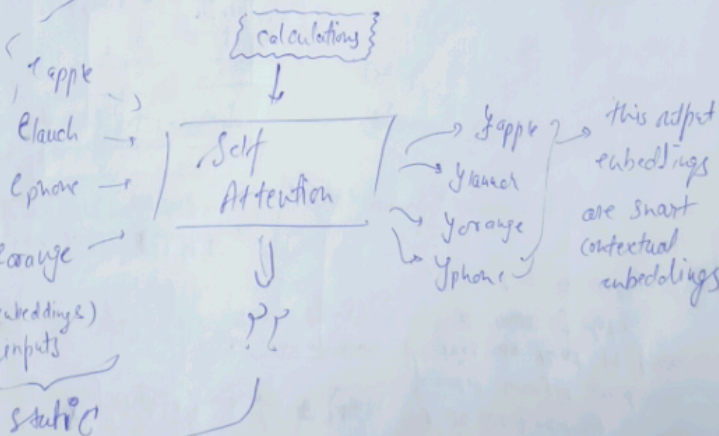
now how to calculate weights?

↳ since these all are vectors in high dimension space, we can find similarities between them by using dot product.

$$e_{bank}^{(new)} = [e_{bank} \cdot e_{money}] e_{money} + [e_{bank} \cdot e_{bank}] e_{bank} + [e_{bank} \cdot e_{grows}] e_{grows}$$

dot product gives similarities as they are already calculated embedding from old technique.

also normalize them, so that it will remain stable. [using softmax]





# points of Self Attention

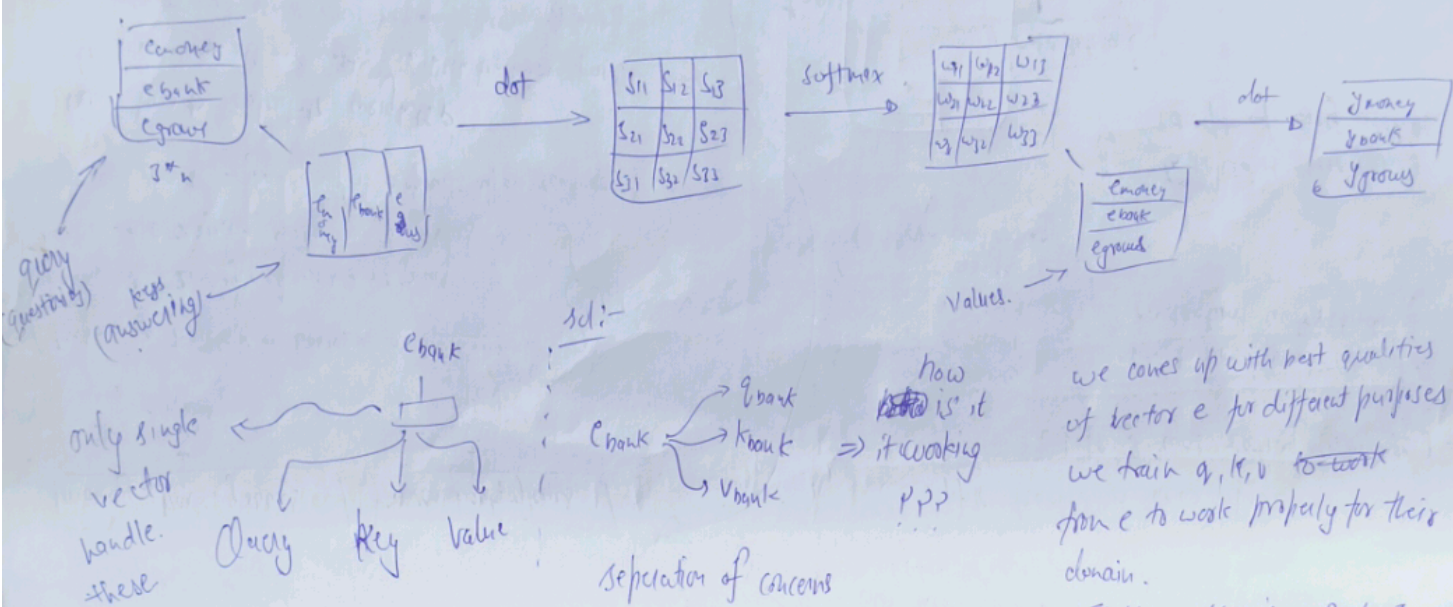
- \* These are parallel processes, so are faster
- \* There are no parameters involved.
- \* It is a general contextual embedding, not task specific

eg: price of cake  $\rightarrow$   $\frac{1}{2}$  for 100 gms  
(actually)  $\rightarrow$  200000 gms

so we need to add weight and bias so that it learn on basis of data instead of general meaning

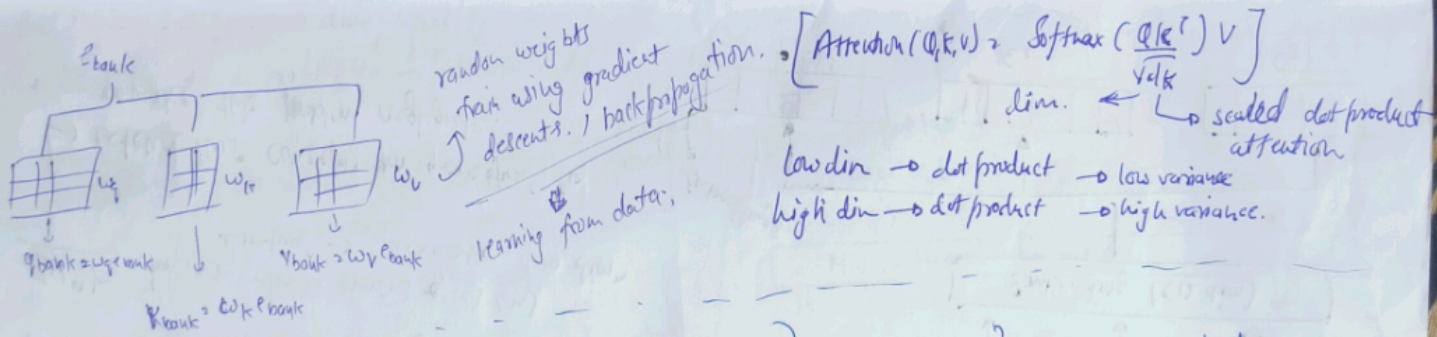
Now have to make it task specific so that it would give better result

fill holes



sol:-

we comes up with best qualities of vector  $e$  for different purposes we train  $q, k, v$  to work from  $e$  to work properly for their domain.  
[Author - Horridge Analog]



$$Attention(q, k, v) = \text{Softmax} \left( \frac{qk^T}{\text{dim}} \right) v$$

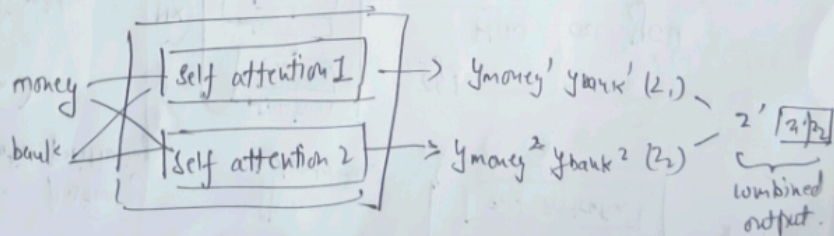
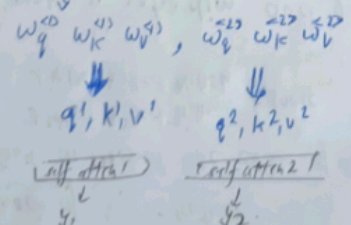
low dim  $\rightarrow$  dot product  $\rightarrow$  low variance  
high dim  $\rightarrow$  dot product  $\rightarrow$  high variance.

eg: "The man saw the astronomer with a telescope"  
 $\rightarrow$  It has two meaning  $\rightarrow$  but the self attention will focus on single meaning/perspective.  
astronomer and telescope are close so they have high similarities as may not be.

Drawback  $\rightarrow$  as in real life NLP is complex.  
solution  $\rightarrow$  to choose more than one self attention  $\rightarrow$  Multi-head attention

## # Multi-Head Self Attention

$\rightarrow$  define 2 or more set of weight matrices



but the shape should be same as money/bank  $\rightarrow$  trainable.

$$(to \text{ fix dim}) \quad 2' \times (w_{\odot}) \rightarrow 2 (y)$$



## # Drawback of Self attention

↳ Can't understand the order of data  
eg: Nitish killed lion  
Lion killed Nitish } some

→ no fix it  
positional encoding came into picture

Sol 1 → make a embedding and give discrete positioning  
↳ problem: unbound, discrete, relative positioning

(we need → bounded, continuous, periodic function)

↳ trigonometric func<sup>n</sup> → let's say  $\sin(n)$

also  $\sin(n) \neq \sin(g)$ ,  $n, g \rightarrow \mathbb{N}$  } no sol<sup>n</sup> exist (distinct)

also it solves all the above problem

however applicable if  $n, g \rightarrow$  small range and precision is large. let's say  $\sin(2) = 0.9063$   
 $\sin(4) = 0.7568$   
 $n \neq g$ , but calc<sup>n</sup> →  $n \approx g$

Sol<sup>n</sup> → func<sup>n</sup>  $f(n) = \sin(n)$  &  $\cos(n)$   
two func<sup>n</sup> → increase two dim<sup>n</sup>

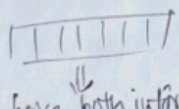
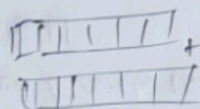
↳ extend this concept for more func<sup>n</sup> to make this problem negligible.

But this will increase dimensionality also.

↳ what we do in real life → "Attention is All you need"

length of positional encoding vector  
= length of embedding vector

instead of concatenate, we do sum.



have both information and same dimension

how to generate these many func<sup>n</sup> p

$$PE(pos, 2i) = \sin(pos / 10000^{2i/d_{model}})$$

$$PE(pos, 2i+1) = \cos(pos / 10000^{2i/d_{model}})$$

## # Normalization

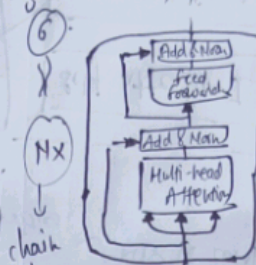
↳ Can't do batch normalization with padding

↳ so we use layer Normalization which works great.

## # Transformer Architecture

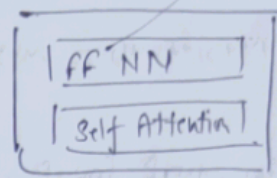
Encoder

↓ (decoder)



chain of encoders.

(input)



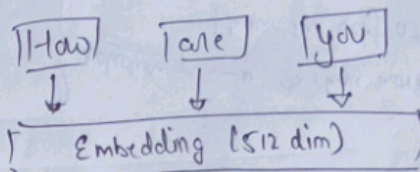
eg

How are you

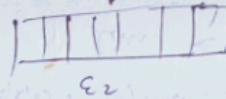
↓

Tokenizer

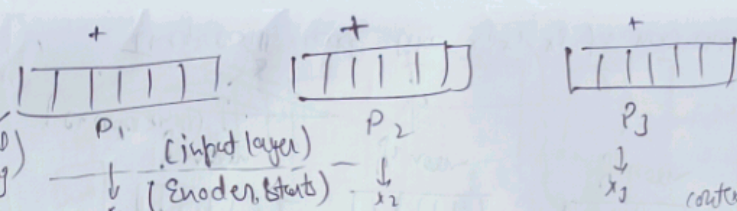
↓



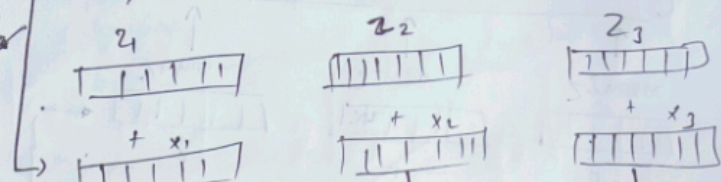
Embeddings



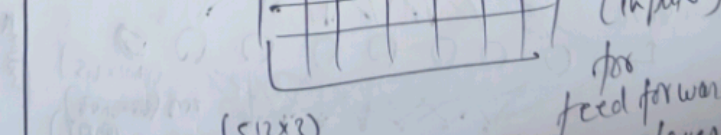
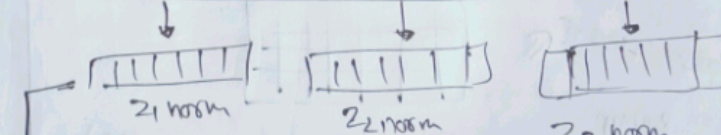
(positional encoding)



Multi-Head Attention



Layer Normalization



(inputs), for feed forward layer



## Some Questions:-

1) Why use relational connections? (grey area).

↳ don't know, my guess is after self attention, the embedding depends upon surrounding more rather than its own information. So to do that we add these residuals to balance the information.

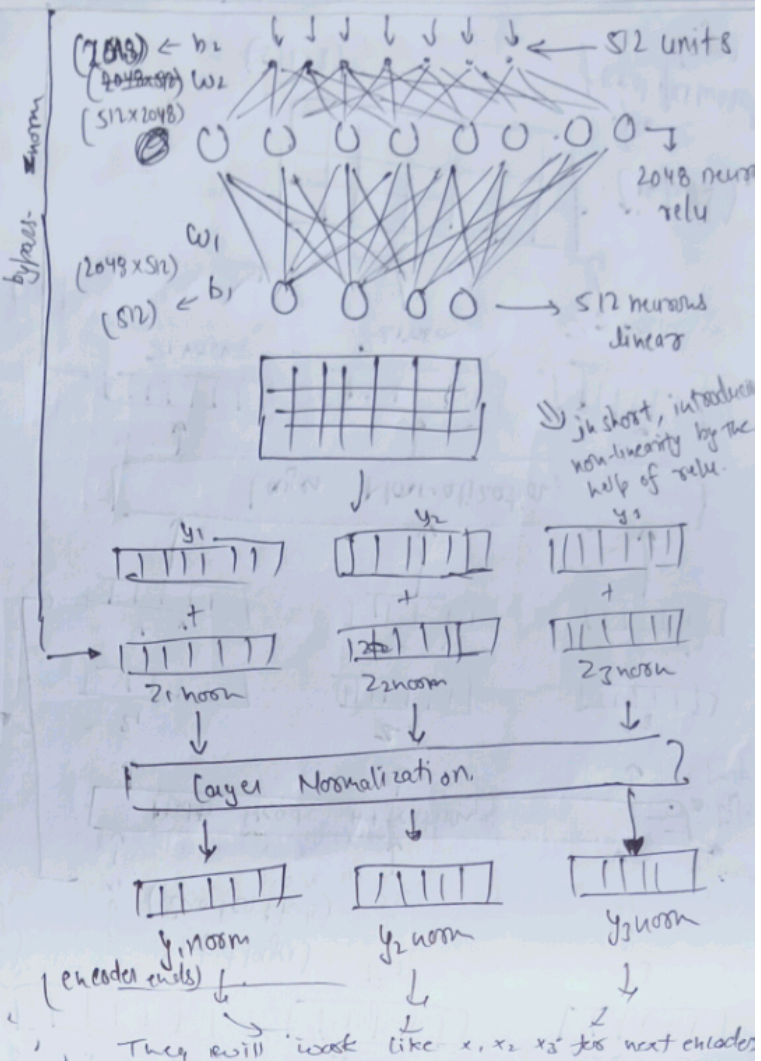
↳ also it provides an alternate path for back propagation, thus solving vanishing gradient in deep NN.

2) Why use feed forward NN? (grey area)

↳ it introduces non-linearity to capture complexities

3) Why use 6 encoder blocks?

↳ experimental.



Same concept with LSTM encoder/decoder.

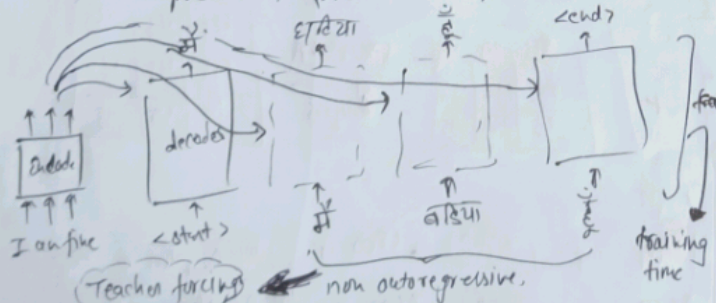
Came from time series concepts.

## # Decoders and Auto regressive Models

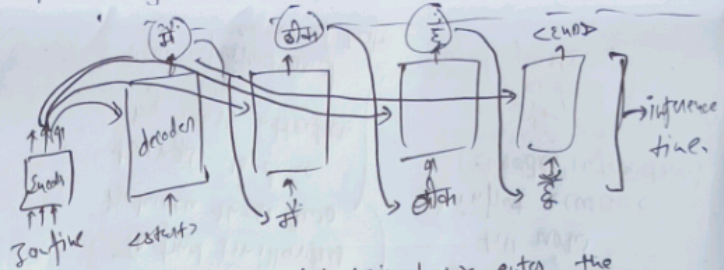
↳ The transformer decoder is autoregressive at inference time and non-autoregressive at training time. (due to masked self attention)

In the context of deep learning, autoregressive models are a class of models that generates data points in a sequence by conditioning each new point on the previously generated points.

↳ for e.g. stock prediction, weather prediction, depends upon previous data points.



↳ since the parameters are initially untrained so we give the correct input whatever the output of previous state was. training just and stable. also can run parallel with teacher forcing



↳ how since the model is trained, we enter the previous state as it will be correct  
↳ also since inference we don't know the output.

## # Problem with Parallel Processing

$$3114_{ce} = 0.8 \cdot 3114_e + 0.1 \cdot 14_e + 0.1 \cdot 22_e$$

initially we don't have these two values, during prediction "cheating" → ok during training but during prediction it is a extra illegal information

## Data leakage

AR  
↳ No data leakage  
↳ Slow

MAR  
↳ fast  
↳ Data leakage

during embedding and self attention.



## # Finding the answers: # Marked Attention

$$\begin{aligned} \text{HLY ce} &= w_{11} * [V_{3114}] + w_{12} * [V_{4114}] + w_{13} * [V_{5114}] \\ \text{HLY ce} &= w_{21} * [V_{3214}] + w_{22} * [V_{4214}] + w_{23} * [V_{5214}] \\ \text{HLY ce} &= w_{31} * [V_{3314}] + w_{32} * [V_{4314}] + w_{33} * [V_{5314}] \end{aligned}$$

$$w_{12} = w_{13} = w_{21} = 0$$

If we can do it we will be good to go.

|                 |                 |                 |
|-----------------|-----------------|-----------------|
| S <sub>11</sub> | S <sub>12</sub> | S <sub>13</sub> |
| S <sub>21</sub> | S <sub>22</sub> | S <sub>23</sub> |
| S <sub>31</sub> | S <sub>32</sub> | S <sub>33</sub> |

Scaling  
1/D  
N<sub>split</sub> (4K)

|                 |                 |                 |
|-----------------|-----------------|-----------------|
| S <sub>11</sub> | S <sub>12</sub> | S <sub>13</sub> |
| S <sub>21</sub> | S <sub>22</sub> | S <sub>23</sub> |
| S <sub>31</sub> | S <sub>32</sub> | S <sub>33</sub> |

|   |    |    |
|---|----|----|
| 0 | -∞ | -∞ |
| 0 | 0  | -∞ |
| 0 | 0  | 0  |

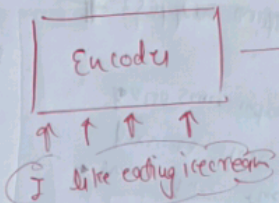
|                 |                 |                 |
|-----------------|-----------------|-----------------|
| S <sub>11</sub> | S <sub>12</sub> | S <sub>13</sub> |
| S <sub>21</sub> | S <sub>22</sub> | S <sub>23</sub> |
| S <sub>31</sub> | S <sub>32</sub> | S <sub>33</sub> |

softmax

|                 |                 |                 |
|-----------------|-----------------|-----------------|
| w <sub>11</sub> | 0               | 0               |
| w <sub>21</sub> | w <sub>22</sub> | 0               |
| w <sub>31</sub> | w <sub>32</sub> | w <sub>33</sub> |

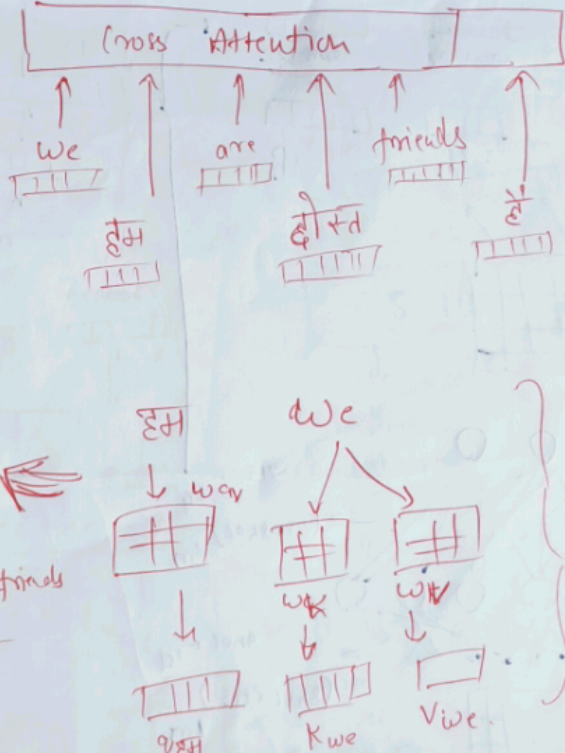
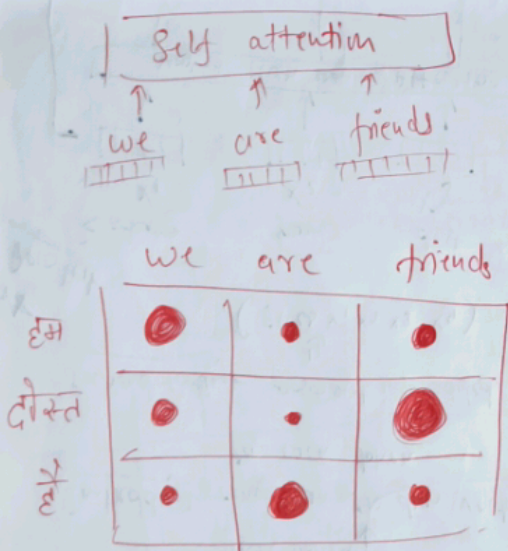
## # Cross Attention

Cross Attention is a mechanism used in transformer architectures, particularly in tasks involving sequence-to-sequence data like translation or summarization. It allows a model to focus on different parts of an input sequence when generating an output sequence.



To find similarities between these two things is done by cross attention  
↓  
very important and core of task.

- what does it depends upon?
- 1) what is generated till now
  - 2) input sentence (encoder information)
- (clouded parts)



for every Hindi word → 1 query vector  
for every english word → 1 key and 1 value vector.  
or  
output seq → query  
input seq → key & value

for whole sentence

$$\begin{aligned} \text{Ce-HI} &= 0.5 \times \text{we} + 0.3 \times \text{are} + 0.2 \times \text{friends} \\ \text{Ce-English} &= \dots \\ \text{Ce-HI} &= \dots \end{aligned}$$

## Use Cases:-

- 1) Image Captioning



# # Architecture of Decoder:- (during training).

Input position  $\rightarrow$  i. Shifting  
 ii. Tokenization  
 iii. Embedding  
 iv. Positional Encoding

Shifting  $\rightarrow$  hum dost hai

$\downarrow$   
 $\langle \text{start} \rangle$  hum dost hai

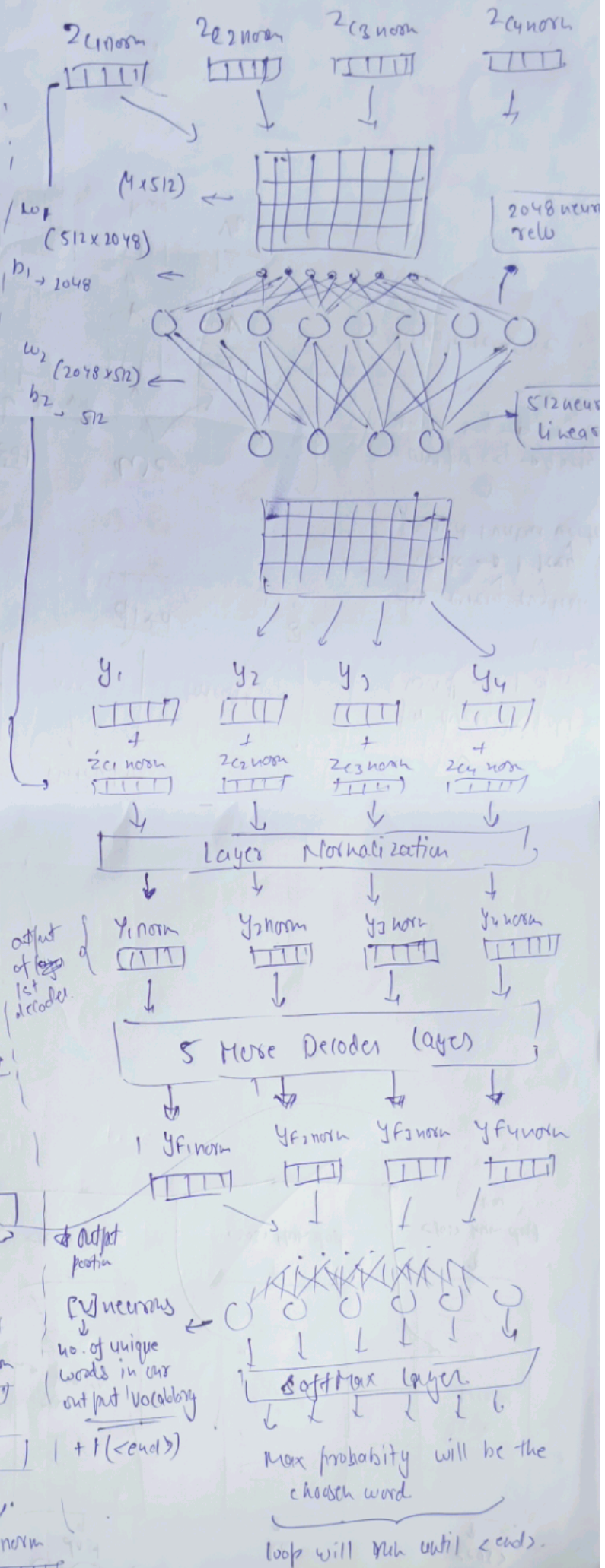
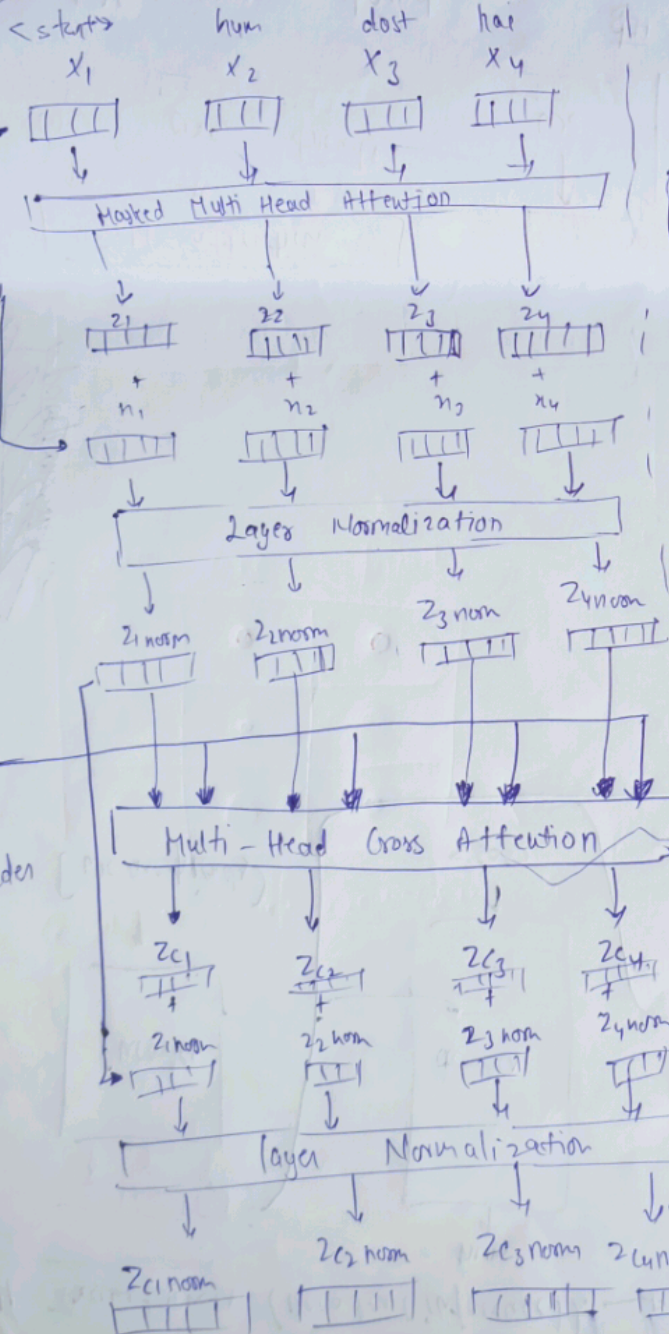
tokenize  $\rightarrow \langle \text{start} \rangle$  [hum] [dost] [hai]  
 create token  $\downarrow$

Embedding: convert in 512 dim vectors for each token.

Positional encoding: positional info added.

$\downarrow$   
 (gives  $x_1, x_2, x_3, x_4$ )

Decoder





# Transformer (Decoder (inference)).  $\rightarrow$  auto regressive  $\rightarrow$  depends upon previous

